

Beyond Dijkstra and the Cut Property

Breaking the Sorting Barrier for Shortest Paths, and the Frontier of Cut Algorithms

Week 5 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why look beyond the syllabus?

- This week you proved Dijkstra's algorithm correct via “greedy stays ahead,” and MST correctness via the **cut property** – both landing on an $O((m + n) \log n)$ bound with a binary heap.
- Two questions a curious student should ask next:
 - ① Dijkstra repeatedly extracts the *minimum*-distance unvisited vertex – which looks a lot like **sorting** the vertices by distance. Sorting n items needs $\Omega(n \log n)$ comparisons in general. Is Dijkstra's $O(m + n \log n)$ therefore an unavoidable “sorting barrier”?
 - ② The cut property says the cheapest edge crossing *any* cut is safe to add. The reverse question – *what is the cheapest cut itself?* (global minimum cut) – is a natural dual problem. How fast can it be found?

Both questions were resolved only very recently

The shortest-path question was open for decades and fell in 2025; a faster algorithm followed within a year. The min-cut question fell in 2024. Every claim below is sourced to a specific, verified paper.

Recap: why Dijkstra looks like sorting

- Dijkstra's algorithm, run with a binary heap, repeatedly does `EXTRACTMIN` to settle vertices in increasing order of $\text{dist}(\cdot)$ – by the time it finishes, it has produced the vertices *sorted* by distance from s .
- Comparison-based sorting needs $\Omega(n \log n)$ comparisons in the worst case – so for decades, $O(m + n \log n)$ looked like it might be an *inherent* barrier for any algorithm that settles vertices one at a time in distance order.
- This is exactly the same shape of question this course asked about Union-Find's $\alpha(n)$ bound in Week 3: *is the best known algorithm also the best possible one?*

The 2025 breakthrough

Theorem (Duan, J. Mao, X. Mao, Shu, Yin, STOC 2025)

On directed graphs with real, non-negative edge weights (comparison-addition model), single-source shortest paths can be solved **deterministically in $O(m \log^{2/3} n)$ time** – strictly beating Dijkstra's $O(m + n \log n)$ on sparse graphs ($m = O(n)$).

- “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths,” *Proc. 57th ACM STOC* (2025), pp. (see DOI). DOI: 10.1145/3717823.3718179; also arXiv:2504.17033.
- **Idea, in one sentence:** instead of a single global priority queue that forces a strict total order on *all* vertices, the algorithm recursively partitions the graph into bounded-hop sub-problems and only partially orders *groups* of vertices – paying for a full sort only within small groups, not across the whole graph.
- This is the first algorithm ever to beat Dijkstra's asymptotic time on sparse directed graphs with real weights – proving the “sorting barrier” was a property of Dijkstra's algorithm, not of the shortest-path *problem*.

An even faster algorithm – eight months later

Theorem (Duan, X. Mao, Shu, Yin, ICALP 2026)

A refinement improves the bound to $O(m\sqrt{\log n} + \sqrt{mn \log n \log \log n})$, which is $O(m\sqrt{\log n \log \log n})$ on sparse graphs – faster still than the 2025 result.

- “A Faster Directed Single-Source Shortest Path Algorithm,” *Proc. 53rd ICALP* (2026); arXiv:2602.07868.
- Submitted April 2025 → improved by February 2026: this is an **active, fast-moving** research frontier, not a settled result – a reminder that “the textbook algorithm” can still be an open research question, exactly as this course’s Week 2 supplement discussed for Dijkstra’s *universal* optimality on specific graph shapes.
- Whether $O(m + n)$ – truly linear, matching BFS – is achievable for real-weighted directed SSSP remains **open**.

Recap: the cut property, and its dual question

- This week's cut property: for any cut (a partition of V into $S, V \setminus S$), the cheapest edge crossing it is safe to add to the MST. Kruskal/Prim exploit this crossing-edge structure *locally*, one cut at a time.
- The **dual** question asks about cuts *globally*: over *all* possible cuts of the graph, which one has the smallest total crossing weight? This is the **global minimum cut** problem – central to network reliability (the cheapest way to disconnect a network).
- Randomized near-linear-time algorithms for this problem existed earlier, but a **deterministic** near-linear-time algorithm for *weighted* graphs was open for years.

The 2024 result

Theorem (Henzinger, J. Li, Rao, D. Wang, SODA 2024)

There is a **deterministic, near-linear-time** algorithm for the global minimum cut in weighted graphs.

- “Deterministic Near-Linear Time Minimum Cut in Weighted Graphs,” *Proc. 2024 ACM-SIAM SODA*, pp. 3089–3139; arXiv:2401.05627.
- Builds on Kawarabayashi & Thorup’s earlier near-linear-time result for *simple* (unweighted) graphs; the core new tool is a clustering procedure that provably *preserves* every minimum cut while shrinking the graph.
- The connection to this week: MST’s cut property and min-cut are both “cut-structure” problems on the same graph – one is easy to exploit locally (MST), the other took until 2024 to pin down fast and deterministically *globally*.

Summary

- ❶ **Dijkstra is not asymptotically optimal.** Duan, Mao, Mao, Shu & Yin (STOC 2025) broke the “sorting barrier” with a deterministic $O(m \log^{2/3} n)$ algorithm for directed real-weighted SSSP; a 2026 follow-up pushed this further to $O(m \sqrt{\log n \log \log n})$ on sparse graphs.
- ❷ **This is an active frontier, not settled history.** Both results postdate this course’s textbook and appeared within the last 16 months – whether truly linear-time SSSP is possible remains open.
- ❸ **The cut property has a global dual.** Henzinger, Li, Rao & Wang (SODA 2024) gave the first deterministic near-linear-time algorithm for global minimum cut in weighted graphs, resolving a problem that had resisted derandomization for years.
- ❹ **The lesson repeats from Week 2 and Week 3:** a clean, decades-old greedy algorithm can still hide open questions about *optimality* – and those questions are actively being resolved today.

Looking ahead

Next week (Kleinberg & Tardos, Chapter 5) leaves greedy design behind for **divide and conquer**: merge sort, recurrences, and the Master theorem – the same style of “prove a tight bound, then ask if it’s optimal” thinking applies there too (as Week 1’s Karatsuba supplement already hinted).